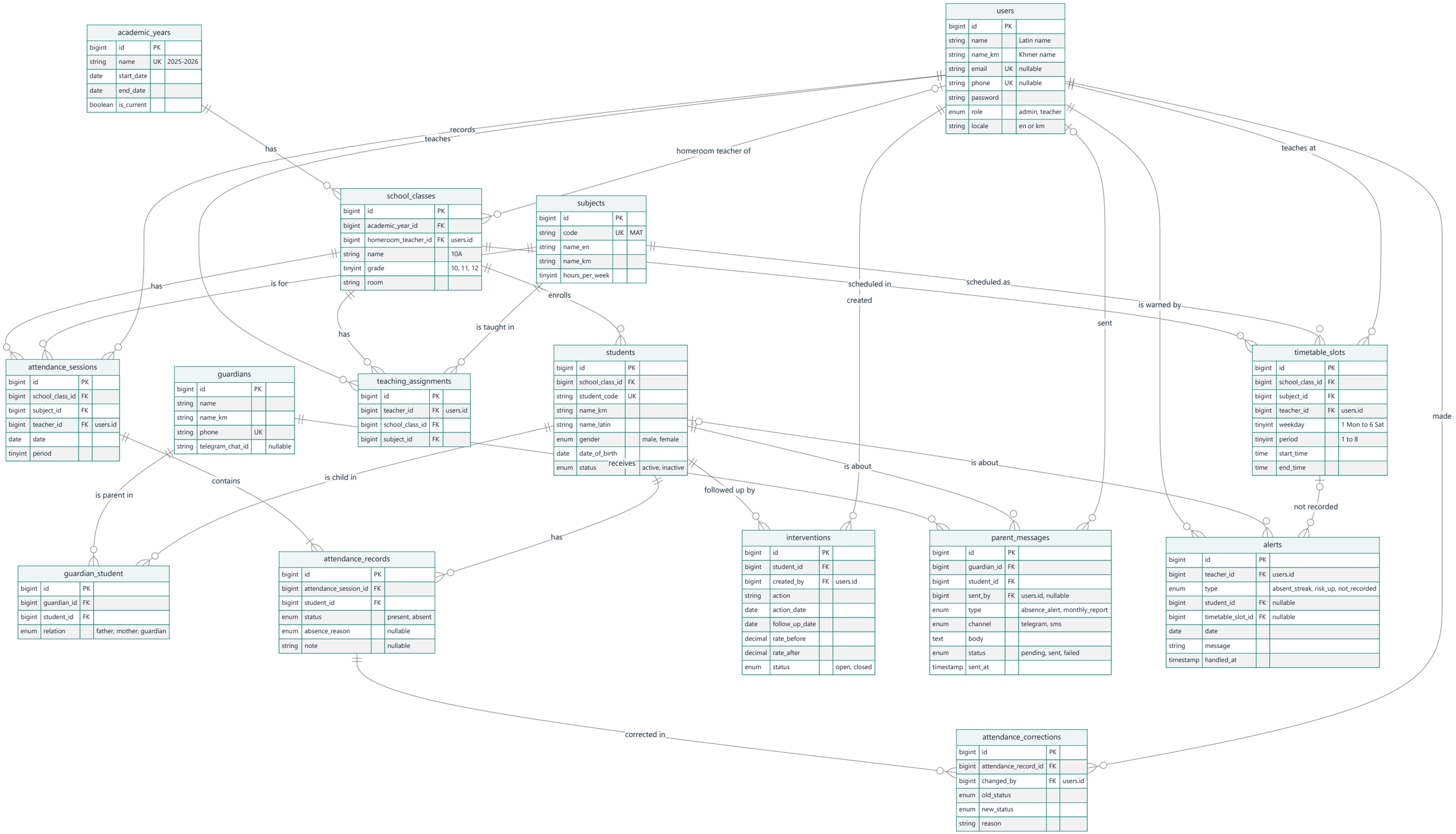
1. ERD: the project tables

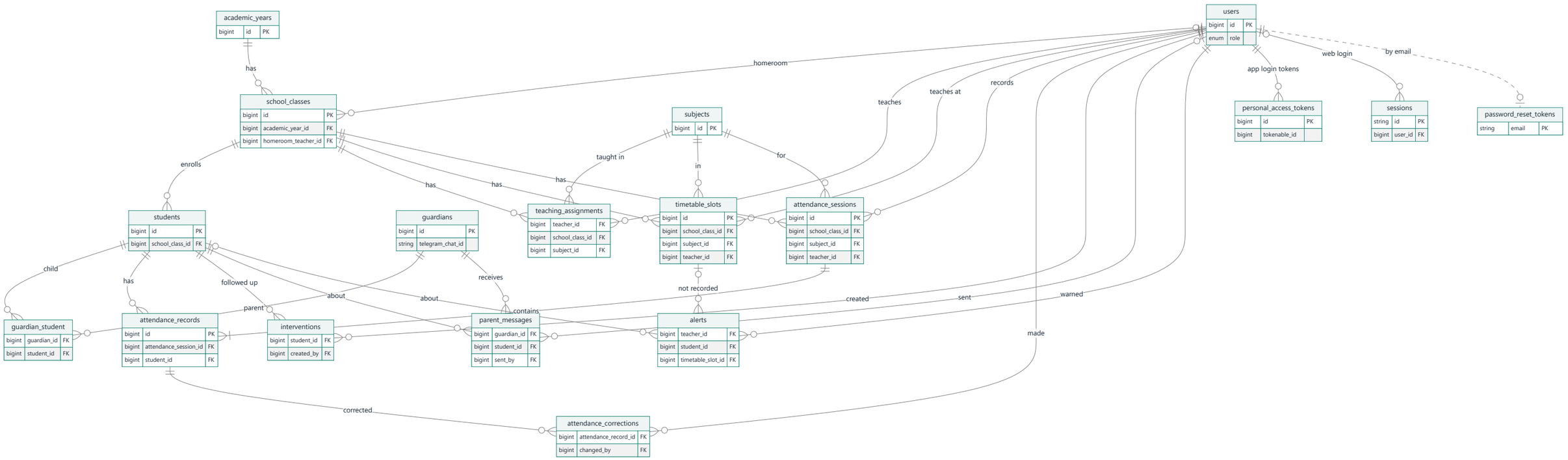
Student Attendance Analytics System · Plan v2.0 · Laravel + MySQL · database **attendance_db**



PK primary key **FK** foreign key **UK** unique **||** exactly one **|o** zero or one **o{** zero or many **|{** one or many Every table also has **created_at** and **updated_at**.

2. Full database ERD: all linked tables

The 15 project tables (key columns only) plus the Laravel tables that link to users: app login tokens, web sessions and password reset



Framework tables drawn: personal_access_tokens, sessions, password_reset_tokens

Not drawn (no links): cache, cache_locks, jobs, job_batches, failed_jobs, migrations. Their columns are on pages 5 and 6.

Dotted line = linked by value (email), no foreign key.

3. Data dictionary: school data

Users, the school structure, students and their parents

Plan v2.0 rules built into the database: only Admin and Teacher log in · parents are contacts that receive Telegram or SMS messages · attendance is only Present or Absent, with an optional absence reason.

users

Column	Type	Description
id	bigint	Primary key
name	varchar	Name in Latin letters
name_km	varchar	Name in Khmer
email	varchar	Unique, optional
phone	varchar	Unique, optional; can be used to log in
password	varchar	Hashed (bcrypt), never plain text
role	enum	admin or teacher
locale	varchar	Preferred language: en or km
email_verified_at	timestamp	Laravel default, optional
remember_token	varchar	For "Remember me"

Only school staff have accounts. The role decides which screens a user can open.

academic_years

Column	Type	Description
id	bigint	Primary key
name	varchar	Unique, e.g. 2025-2026
start_date	date	First school day
end_date	date	Last school day
is_current	boolean	Only one year is current

school_classes

Column	Type	Description
id	bigint	Primary key
academic_year_id	bigint	FK → academic_years.id
homeroom_teacher_id	bigint	FK → users.id, optional
name	varchar	e.g. 10A
grade	tinyint	10, 11 or 12
room	varchar	Room number

Unique: (academic_year_id, name). Named school_classes because "class" is a reserved word in PHP.

subjects

Column	Type	Description
id	bigint	Primary key
code	varchar	Unique short code, e.g. MAT
name_en	varchar	English name
name_km	varchar	Khmer name
hours_per_week	tinyint	Lessons per week

students

Column	Type	Description
id	bigint	Primary key
school_class_id	bigint	FK → school_classes.id
student_code	varchar	Unique, e.g. S-10A-001
name_km	varchar	Khmer name
name_latin	varchar	Latin name
gender	enum	male, female
date_of_birth	date	Optional
status	enum	active, inactive (left the school)

Students do not log in. They are records managed by the admin.

guardians

Column	Type	Description
id	bigint	Primary key
name	varchar	Latin name
name_km	varchar	Khmer name
phone	varchar	Unique; used for SMS
telegram_chat_id	varchar	Set when the parent joins the school's Telegram bot

Parents are contacts only. Telegram if they joined the bot, otherwise SMS.

guardian_student

Column	Type	Description
id	bigint	Primary key
guardian_id	bigint	FK → guardians.id
student_id	bigint	FK → students.id
relation	enum	father, mother, guardian

Pivot table: one parent can have many children, one child can have many parents. Unique: (guardian_id, student_id).

teaching_assignments

Column	Type	Description
id	bigint	Primary key
teacher_id	bigint	FK → users.id (role teacher)
school_class_id	bigint	FK → school_classes.id
subject_id	bigint	FK → subjects.id

Who teaches which subject in which class. A teacher sees only these classes. Unique: (school_class_id, subject_id).

timetable_slots

Column	Type	Description
id	bigint	Primary key
school_class_id	bigint	FK → school_classes.id
subject_id	bigint	FK → subjects.id
teacher_id	bigint	FK → users.id
weekday	tinyint	1 Monday … 6 Saturday
period	tinyint	Lesson number, 1 to 8
start_time / end_time	time	e.g. 07:00 to 08:00

The weekly timetable. Gives the teacher "today's schedule". Unique: (class, weekday, period) and (teacher, weekday, period), so nobody is in two places at once.

4. Data dictionary: attendance, analytics and parent contact

Lessons, records, the audit log, follow-ups, parent messages and early warnings

attendance_sessions

Column	Type	Description
id	bigint	Primary key
school_class_id	bigint	FK → school_classes.id
subject_id	bigint	FK → subjects.id
teacher_id	bigint	FK → users.id (who recorded it)
date	date	Day of the lesson
period	tinyint	Lesson period, 1 to 8

One lesson where attendance was taken. Unique: (school_class_id, date, period), so the same lesson cannot be recorded twice.

attendance_records

Column	Type	Description
id	bigint	Primary key
attendance_session_id	bigint	FK → attendance_sessions.id
student_id	bigint	FK → students.id
status	enum	present or absent, nothing else
absence_reason	enum	Optional: not_given, sick, family_matter, school_activity, other
note	varchar	Optional

One row per student per lesson. Unique: (attendance_session_id, student_id). All analytics come from this table. Rate = present ÷ all records.

attendance_corrections

Column	Type	Description
id	bigint	Primary key
attendance_record_id	bigint	FK → attendance_records.id
changed_by	bigint	FK → users.id (who made the change)
old_status	enum	present or absent, before
new_status	enum	present or absent, after
reason	varchar	Why it was changed
created_at	timestamp	When the change was made

Audit log. Teachers correct on the same day; the admin corrects past days. Records are never changed silently.

interventions

Column	Type	Description
id	bigint	Primary key
student_id	bigint	FK → students.id
created_by	bigint	FK → users.id
action	varchar	e.g. Parent message (Telegram), meeting with parent
notes	text	Optional
action_date	date	When the action was taken
follow_up_date	date	When to check again
rate_before / rate_after	decimal	Attendance rate before, and when closed
status	enum	open or closed
closed_at	timestamp	When it was closed

Follow-up actions for at-risk students. rate_before and rate_after show whether the action helped.

parent_messages

Column	Type	Description
id	bigint	Primary key
guardian_id	bigint	FK → guardians.id
student_id	bigint	FK → students.id
sent_by	bigint	FK → users.id; empty = sent by the system
type	enum	absence_alert or monthly_report
channel	enum	telegram or sms
body	text	Message text, Khmer then English
status	enum	pending, sent, failed
error	varchar	Why sending failed
sent_at	timestamp	When it was delivered

Every message to a parent, so the school can prove what was sent and when.

alerts

Column	Type	Description
id	bigint	Primary key
teacher_id	bigint	FK → users.id (who should see it)
type	enum	absent_streak, risk_up, not_recorded
student_id	bigint	FK → students.id, for student alerts
timetable_slot_id	bigint	FK → timetable_slots.id, for "not recorded"
date	date	Day of the alert
message	varchar	e.g. Absent 3 school days in a row
handled_at	timestamp	When the teacher handled it
handled_by	bigint	FK → users.id

Early warnings. Risk levels themselves are calculated, not stored: Critical (<70% or 5+ days in a row), High (70–79%, 3–4 days or down 10+ points), Medium (80–89% or down 5–9), Low.

5. Data dictionary: framework tables (part 1)

Tables created by Laravel and Laravel Sanctum. The project uses them but does not design them.

personal_access_tokens (Sanctum)

Column	Type	Description
id	bigint	Primary key
tokenable_type	varchar	Owner model, here App\Models\User
tokenable_id	bigint	Owner id → users.id
name	varchar	Device name, e.g. "android"
token	varchar(64)	Unique, hashed token
abilities	text	What the token may do
last_used_at	timestamp	Last request from the app
expires_at	timestamp	Optional expiry

Lets the teacher's Android app log in. Added in Step 3 with Laravel Sanctum. Linked to users by a polymorphic key, not a foreign key.

sessions

Column	Type	Description
id	varchar	Primary key, session id
user_id	bigint	Logged-in user, optional
ip_address	varchar(45)	Browser IP address
user_agent	text	Browser name
payload	longtext	Session data
last_activity	int	Unix time of last request

Keeps web users logged in between pages.

password_reset_tokens

Column	Type	Description
email	varchar	Primary key, the user's email
token	varchar	Hashed reset code
created_at	timestamp	When the code was made

Used only for "Forgot password".

cache

Column	Type	Description
key	varchar	Primary key
value	mediumtext	Saved value
expiration	int	When it expires

Stores results such as dashboard numbers so pages load faster.

cache_locks

Column	Type	Description
key	varchar	Primary key
owner	varchar	Who holds the lock
expiration	int	When the lock ends

Stops two processes from changing the same cached item at once.

migrations

Column	Type	Description
id	int	Primary key
migration	varchar	Migration file name
batch	int	Which run created it

Laravel's record of which table files have been run. Never edited by hand.

6. Data dictionary: framework tables (part 2)

Background jobs. Sending parent messages runs as a background job, so a slow Telegram or SMS service never slows down taking attendance.

jobs

Column	Type	Description
id	bigint	Primary key
queue	varchar	Queue name
payload	longtext	The task to run
attempts	tinyint	Times tried
reserved_at	int	When a worker took it
available_at	int	When it may run
created_at	int	When it was added

Tasks waiting to run, for example "send the absence alert for Sok Dara".

job_batches

Column	Type	Description
id	varchar	Primary key
name	varchar	Batch name
total_jobs / pending_jobs / failed_jobs	int	Progress counters
failed_job_ids	longtext	Which jobs failed
options	mediumtext	Batch settings
cancelled_at / created_at / finished_at	int	Batch times

Groups of tasks, for example all monthly reports at the end of the month.

failed_jobs

Column	Type	Description
id	bigint	Primary key
uuid	varchar	Unique job id
connection / queue	text	Where it ran
payload	longtext	The task
exception	longtext	The error message
failed_at	timestamp	When it failed

Tasks that failed, kept so they can be retried.

7. Relationships and table summary

Every link between tables in words, and the full list of tables

Relationships

#	Parent	Child (foreign key)	Type	Meaning
1	academic_years	school_classes.academic_year_id	1 : N	A school year has many classes
2	users	school_classes.homeroom_teacher_id	0..1 : N	A teacher can be homeroom teacher of classes
3	school_classes	students.school_class_id	1 : N	A class has many students
4	guardians	guardian_student.guardian_id	1 : N	A parent is linked to children
5	students	guardian_student.student_id	1 : N	A child is linked to parents
—	guardians ↔ students through guardian_student		M : N	Many parents, many children
6	users	teaching_assignments.teacher_id	1 : N	A teacher has many assignments
7	school_classes	teaching_assignments.school_class_id	1 : N	A class has many subjects taught
8	subjects	teaching_assignments.subject_id	1 : N	A subject is taught in many classes
9	school_classes	timetable_slots.school_class_id	1 : N	A class has a weekly timetable
10	subjects	timetable_slots.subject_id	1 : N	A subject appears in many slots
11	users	timetable_slots.teacher_id	1 : N	A teacher has a weekly schedule
12	school_classes	attendance_sessions.school_class_id	1 : N	A class has many lessons recorded
13	subjects	attendance_sessions.subject_id	1 : N	A subject has many lessons
14	users	attendance_sessions.teacher_id	1 : N	A teacher records many lessons
15	attendance_sessions	attendance_records.attendance_session_id	1 : N (1+)	A lesson has one record per student
16	students	attendance_records.student_id	1 : N	A student has many attendance records
17	attendance_records	attendance_corrections.attendance_record_id	1 : N	A record can be corrected several times
18	users	attendance_corrections.changed_by	1 : N	Who made each correction
19	students	interventions.student_id	1 : N	A student can have several follow-ups
20	users	interventions.created_by	1 : N	Who started each follow-up
21	guardians	parent_messages.guardian_id	1 : N	A parent receives many messages
22	students	parent_messages.student_id	1 : N	Which child a message is about
23	users	parent_messages.sent_by	0..1 : N	Who sent it; empty = the system
24	users	alerts.teacher_id	1 : N	A teacher receives alerts
25	students	alerts.student_id	0..1 : N	Which student an alert is about
26	timetable_slots	alerts.timetable_slot_id	0..1 : N	Which lesson was not recorded
27	users	alerts.handled_by	0..1 : N	Who handled the alert
28	users	personal_access_tokens.tokenable_id	1 : N	A teacher can log in on several phones
29	users	sessions.user_id	0..1 : N	Web login sessions

All 24 tables

#	Table	Group	Status
1	users	Project	Built ✓
2	academic_years	Project	Built ✓
3	school_classes	Project	Built ✓
4	subjects	Project	Built ✓
5	students	Project	Built ✓
6	guardians	Project	Built ✓
7	guardian_student	Project	Built ✓
8	teaching_assignments	Project	Built ✓
9	timetable_slots	Project	Built ✓
10	attendance_sessions	Project	Built ✓
11	attendance_records	Project	Built ✓
12	attendance_corrections	Project	Built ✓
13	interventions	Project	Built ✓
14	parent_messages	Project	Built ✓
15	alerts	Project	Built ✓
16	personal_access_tokens	Framework	Step 3 (Sanctum)
17	sessions	Framework	Laravel default ✓
18	password_reset_tokens	Framework	Laravel default ✓
19	cache	Framework	Laravel default ✓
20	cache_locks	Framework	Laravel default ✓
21	jobs	Framework	Laravel default ✓
22	job_batches	Framework	Laravel default ✓
23	failed_jobs	Framework	Laravel default ✓
24	migrations	Framework	Laravel default ✓

✓ = already in attendance_db. personal_access_tokens is created in Step 3.